SHL Assessment Recommender Agent - Approach & Evaluation

1. Why Deterministic Orchestration?

Early iterations of LLM-based agents often used 'ReAct' loops or massive monolithic prompts. These models proved brittle, prone to hallucinating assessment URLs, and difficult to test.

By switching to a Deterministic Orchestration pattern, we abstracted the logic out of the LLM. The LLM is relegated solely to synthesizing human-readable text and parsing complex intent. All state tracking, routing, and retrieval decisions are strictly executed in Python. This guarantees that the system never hallucinates a URL, never recommends a product outside the catalog, and always adheres to the JSON response schema.

2. Hybrid Retrieval Design

Finding the right assessment requires both semantic understanding (e.g., 'customer service' matching 'client relations') and exact keyword matching (e.g., 'Java', 'Docker').

- Semantic: Dense embeddings via SentenceTransformers match vague job descriptions to assessment descriptions.
- Keyword: BM25 (or TF-IDF overlap fallback) ensures that specific technical skills explicitly requested are weighted heavily.
- Post-Filtering: The engine automatically bumps required baseline assessments (like OPQ32r or Verify G+) into the results if personality or cognitive requirements are flagged by the intent agent.

3. Stateless Reconstruction

The API is completely stateless. Every POST /chat provides the full messages array. Agent 2 (Intent) parses this array iteratively to build the ConversationState (tracking role, skills, exclusions, etc.). This ensures the evaluator can jump to turn 4 immediately without previous DB state, and allows highly concurrent cloud deployments.

4. Guardrails & Hallucination Prevention

- Agent 10 (Scope Guard): Employs explicit regex and heuristic checks before any processing to trap prompt injections, legal queries, or non-hiring chat.
- Catalog Grounding: The LLM is provided the exact list of retrieved assessments in its system prompt and explicitly instructed not to invent names. Even if it disobeys, the final JSON output extracts the URLs straight from the Python retrieval list, not the LLM's text output.

SHL Assessment Recommender Agent - Approach & Evaluation

5. Evaluation Methodology & Results

Improvement was measured empirically using the evaluate.py harness which tests:
1. Recall@10: Did the expected assessment IDs land in the top 10 deterministic recommendations?
2. Behavior Probes: 10 programmatic checks verifying that the agent refuses vaguely worded turn-1 queries, honors edits (e.g., 'drop OPQ'), and adheres to schema.

Baseline Evaluation Results (from final_eval_report.txt):

Conversation Trace	Recall@10 Score	Conversation Trace	Recall@10 Score
C1.md (Leadership Selection)	0.33 (Expected 3, Got 10)	C6.md (Graduate Scenario)	1.00 (Expected 2, Got 10)
C2.md (Senior Rust Dev)	0.00 (Expected 5, Got 10)	C7.md (Sales Executives)	0.00 (Expected 5, Got 10)
C3.md (Technical Support)	0.75 (Expected 4, Got 10)	C8.md (Business Analysts)	0.20 (Expected 5, Got 10)
C4.md (Retail Sales Associate)	0.20 (Expected 5, Got 10)	C9.md (Project Managers)	0.00 (Expected 7, Got 10)
C5.md (Data Analyst)	0.60 (Expected 5, Got 10)	C10.md (Graduate Trainee)	0.00 (Expected 2, Got 10)
Mean Recall@10	**0.308**	**Behavior Probes Score**	**10 / 10 (100% Pass)**

6. Tradeoffs & Failure Handling

- Tradeoff: Using a local SentenceTransformer model requires memory overhead and slightly increases cold-start times compared to using an external embeddings API. However, it completely eliminates embedding API costs.
- Graceful Degradation: During evaluation, the external Groq API frequently hit 429 Too Many Requests. The system caught these exceptions and seamlessly fell back to returning the retrieved assessments as a plain-text list. This ensured the evaluator didn't crash and the Mean Recall metrics could still be computed.

7. What Did Not Work

- Pure Vector Search: Relying solely on dense embeddings failed miserably for highly technical roles. The model couldn't differentiate between 'Java' and 'JavaScript' accurately because they live close in semantic space. Integrating exact keyword overlap was mandatory.
- Relying on the LLM to output the JSON: Early attempts to have the LLM output the final JSON schema failed roughly 5% of the time due to missing brackets or hallucinated keys. Moving the schema assembly to Python make_response() solved this permanently.